LECTURE 24

Object Storage: Multipart Uploads, Downloads & Streaming

Backend Engineering from First Principles

Detailed, GitHub-ready study notes based on the uploaded lecture transcript

Topics: multipart uploads, resumability, ETags and checksums, pre-signed URLs, CDN caching, HTTP ranges, HLS/DASH, and storage costs.

Backend Engineering from First Principles

Detailed notes based on the uploaded lecture transcript

Learning objectives

By the end of this lecture, you should be able to:

- Explain why a single `PUT` request is unsuitable for very large files.
- Describe the three core phases of multipart upload and the role of the upload ID and ETags.
- Choose a sensible multipart part size and concurrency limit.
- Explain the multipart ETag trap and how to verify end-to-end integrity.
- Prevent storage leaks caused by abandoned multipart uploads.
- Design a browser-to-object-storage upload flow using pre-signed URLs.
- Compare public/CDN delivery with pre-signed GET URLs.
- Explain HTTP range requests and how they enable seeking, resuming, and parallel downloads.
- Describe segmented video streaming with HLS/DASH and adaptive bitrate.
- Identify the main object-storage cost dimensions.

---

1. Why a single PUT stops working for large files

A single-request upload can be perfectly adequate for smaller objects, but it becomes fragile and limiting as file size grows. The lecture identifies four reasons.

1.1 Hard size limit

The object-storage API imposes a maximum size for a single `PUT` request. In the lecture's S3 example, the limit is approximately **5 GB**. This is a service/API constraint, not a limit imposed by HTTP itself or by your application code.

Products that accept large videos, CAD files, database dumps, wedding footage, or drone footage therefore need a different upload design.

1.2 All-or-nothing behavior

A single PUT is one large operation. If the connection drops near the end, the request fails and the client generally has to start the upload again from byte zero.

That is especially painful on mobile networks:

- Uploads can take many minutes.

- Temporary network interruptions are normal.
- A nearly completed upload can be lost because of a brief connectivity issue.
- There is no natural per-chunk retry boundary.

1.3 Throughput limits of one TCP connection

A single TCP stream may fail to use all the available network capacity. Throughput is influenced by the TCP window and round-trip time, and packet loss can reduce the congestion window.

The lecture's central intuition:

- One connection has one congestion-control state.
- Several independent connections can make progress independently.
- If one stream slows down, the others need not stop with it.

Download managers use a similar idea: they request different ranges of the same file over multiple connections. They do not magically increase the physical capacity of the internet connection; they can use the available capacity more effectively.

1.4 Poor user experience

A single large request offers limited control:

- Progress reporting can be coarse.
- Pause/resume is difficult.
- A closed tab or interrupted connection can force a restart.
- Retry work is expensive because the entire request is the retry unit.

Conclusion: large-file uploads need smaller independently retryable pieces, parallel transfer, and a way to assemble the final object.

---

2. Multipart upload: the three core calls

Multipart upload divides one logical object into independently uploaded parts. The object is not visible at its final key until the upload is completed.

2.1 Create multipart upload

The client-side application asks its backend to initiate an upload. The backend calls the object-storage service with information such as:

- Bucket
- Object key

- Content type and relevant metadata

The service returns an **upload ID**.

Think of the upload ID as identifying a temporary staging area:

- Uploaded parts accumulate under this upload ID.
- The final object does not yet exist at the destination key.
- A GET for the final key will not return the completed object before the completion operation.

2.2 Upload part

For every chunk, the client sends:

- Upload ID
- Part number
- Part bytes

Each part upload is an independent HTTP request. Therefore:

- Parts can be uploaded in parallel.
- Parts can arrive out of order.
- A failed part can be retried without retransmitting successful parts.
- A successful part returns an **ETag** that the client must retain.

The part number identifies the chunk's position in the final object. The ETag is needed later when the upload is completed.

2.3 Complete multipart upload

The completion request contains:

- Upload ID
- The ordered list of part numbers and their corresponding ETags

The object-storage service finalizes the object using the uploaded parts. In the lecture's explanation, completion is primarily a metadata operation: the service records how the stored parts form the final object rather than making the application download and re-upload all bytes through its own backend.

This is why the final completion call can be fast even for a very large object.

Multipart lifecycle at a glance

```
CreateMultipartUpload
|
v
Upload ID
|
```

```
+---- Upload part 1 ----> ETag 1
+---- Upload part 2 ----> ETag 2
+---- Upload part 3 ----> ETag 3
+---- ... in parallel / any order
|
v
CompleteMultipartUpload(upload ID, ordered parts + ETags)
|
v
Final object is available at its key
```

---

3. Part sizing and the 10,000-part cap

Multipart upload has a maximum of **10,000 parts**. Part size is therefore an architectural decision.

3.1 Why a fixed tiny part size can fail

If every part is 5 MB:

- 10,000 parts × 5 MB = 50,000 MB, or roughly 50 GB using decimal units.
- A 60 GB file would exceed the part-count cap before all bytes were uploaded.
- The upload could fail after a large amount of data and time had already been spent.

The key lesson is to calculate part size based on the total file size instead of blindly using one tiny default.

3.2 A practical sizing rule

The lecture suggests:

1. Pick a default/floor part size based on typical product usage, such as 16 MB or 32 MB.
2. Calculate the minimum part size needed to stay within 10,000 parts.
3. Use whichever is larger.

Conceptually:

```
required_part_size = ceil(file_size / 10,000)
chosen_part_size = max(default_part_size, required_part_size)
part_count = ceil(file_size / chosen_part_size)
```

The speaker describes 16–32 MB as a useful floor and roughly 16–64 MB as a practical range for many use cases. The exact value should be validated against the storage provider's current multipart constraints and the product's file-size distribution.

3.3 Why not make parts enormous?

A part is the retry unit. If a 500 MB part fails, the client must resend that whole part. If a 16–32 MB part fails, the retry is much cheaper.

Trade-off:

- **Smaller parts:** cheaper retries and finer progress, but more requests and part metadata.
- **Larger parts:** fewer requests, but more data must be retransmitted on failure.

Choose a size that balances request overhead, retry cost, memory, and the 10,000-part limit.

---

4. The multipart ETag trap

ETags are useful in multipart upload, but they should not automatically be treated as a whole-file content hash.

4.1 Single PUT versus multipart ETag

The lecture explains that a normal single PUT commonly returns an ETag corresponding to an MD5 of the object contents in the basic case.

For multipart uploads, the returned ETag is different. It is commonly derived from the part-level checksums and includes a suffix indicating the number of parts (for example, `-42`).

Therefore, this naïve integrity check is unsafe for multipart objects:

```
local_md5(file) == multipart_object_etag
```

The values are not necessarily comparable.

4.2 Better integrity verification

Use an explicit checksum mechanism:

- Use the object-storage provider's supported checksum feature (the lecture mentions CRC32C or SHA-256 options in modern S3 SDK workflows).
- Or calculate a SHA-256 hash on the client, store it as object metadata, and verify it in a background job after upload.

A verified content hash can also support duplicate detection/deduplication workflows, provided the application designs that behavior carefully.

Important nuance: ETag behavior can depend on provider, upload mode, encryption, and implementation. Treat ETag as the provider's entity tag/part-completion token unless the provider explicitly documents the checksum semantics you need.

---

5. Abandoned multipart uploads: invisible storage cost

A multipart upload can be initiated and partially filled without ever being completed.

Example from the lecture:

- 300 parts
- Approximately 32 MB per part
- Roughly 10 GB sitting in the staging area
- The user closes the tab before completion or abort

Those uploaded parts can continue consuming billable storage even though the final object does not appear in ordinary bucket object listings. The application may not notice the abandoned bytes by looking only at the bucket's normal object list.

Prevention: lifecycle abort rule

Configure a bucket lifecycle rule to abort incomplete multipart uploads after a defined period. The lecture recommends **7 days** as a practical default.

Operational checklist:

- Apply the rule to every production bucket that supports multipart uploads.
- Monitor incomplete multipart uploads where the provider exposes that information.
- Explicitly abort when the user cancels an upload, where practical.
- Do not rely only on normal object listing to find abandoned parts.

---

6. Production architecture: pre-signed URLs + multipart upload

The recommended architecture keeps large file bytes out of the application server's request path.

6.1 Initiation

1. Browser calls the backend, for example `POST /uploads/init`.
2. It sends file name, size, and content type.
3. Backend authenticates the user and authorizes the operation.
4. Backend validates file constraints and computes part size/count.
5. Backend initiates multipart upload with object storage.
6. Backend records an upload row in its database with status such as `pending`.

7. Backend returns the upload ID, part plan, and pre-signed URLs (or a way to request them in batches).

The database row records the application's intent and ownership. Object storage handles the bytes; the database tracks application-level file state.

6.2 Browser uploads parts directly

The browser:

- Slices the file into chunks.
- Sends each chunk directly to object storage using its pre-signed URL.
- Uploads several parts concurrently.
- Stores each successful part number and ETag.

The lecture suggests starting around **4–6 concurrent part uploads** as a practical balance. Tune this based on device/network conditions and observed performance.

6.3 Completion and verification

1. Browser calls the backend's completion endpoint.
2. It submits the upload ID and the list of part numbers/ETags.
3. Backend calls the storage provider's multipart completion API.
4. Backend performs a `HEAD` request on the final object.
5. Backend verifies expected properties such as size and content type (and any other trusted metadata/checksum).
6. Backend changes the database row from `pending` to `completed`/`ready`.
7. Backend may publish a `file.uploaded` event for downstream jobs, audit logging, notifications, or UI updates.

Do not mark a file ready merely because the browser claims that it uploaded all parts. Verify the finalized object server-side.

Architecture sequence

```
Browser Backend / DB Object Storage
| | |
|-- POST /uploads/init->| |
| |-- create multipart ------>|
| |<-- upload ID -------------|
| |-- save pending row |
|<-- part plan + URLs ---| |
| |
|-- PUT part 1 ------------------------------------>|
|-- PUT part 2 ------------------------------------>|
|-- PUT part 3 ------------------------------------>|
| (parallel, direct to storage) |
```

```
|<-- ETags ----------------------------------------|
| |
|-- POST /uploads/complete ----------------------->|
| |-- complete multipart ---->|
| |<-- completed -------------|
| |-- HEAD object ------------>|
| |<-- metadata --------------|
| |-- mark DB row completed |
|<-- success / event ----| |
```

6.4 Do not pre-sign 10,000 URLs in one response

For very large uploads, generating and returning every URL at once can create a huge JSON response. Instead, issue URLs in batches (the lecture gives 100 as an example), as the browser progresses through the parts.

6.5 Browser memory and file slicing

The browser's `File` object supports slicing. A slice produces a `Blob` representing a section of the file; the browser does not need to load the entire multi-gigabyte file into JavaScript memory up front.

A useful approximation from the lecture:

```
in-flight upload memory ≈ part_size × concurrency
```

For example, 16 MB parts with five concurrent uploads implies roughly 80 MB of part payloads in flight, rather than loading a 2 GB file all at once. Actual browser memory use also depends on buffering and implementation details.

6.6 CORS and readable ETags

The browser must be allowed to read the ETag response header. Configure the bucket's CORS policy to expose the `ETag` header; otherwise, JavaScript may be unable to access it even when the storage service returns it.

CORS is a browser access-control mechanism, not a replacement for authorization. The pre-signed URL still needs to be scoped and protected appropriately.

---

7. Resuming an interrupted multipart upload

Multipart upload creates a natural resume mechanism because completed parts remain associated with the upload ID.

After a page refresh or network interruption:

1. Browser asks the backend whether an upload is already in progress for the relevant file/user.

2. Backend looks up the pending upload record.
3. Backend calls the provider's list-parts operation for that upload ID.
4. Backend identifies which parts already exist.
5. Backend returns fresh pre-signed URLs for only the missing parts.
6. Browser uploads the missing parts and completes the upload.

The client resumes at the missing part(s), rather than starting the entire file again.

A production design should also define:

- How the application identifies a resumable upload (user, intended object, file size, checksum/fingerprint, and expiry).
- What happens when an upload ID expires or is aborted.
- How concurrent tabs/devices are prevented from completing conflicting uploads.
- How cancellation and abandoned-upload cleanup work.

---

8. Downloads: do not proxy large files through the backend

The lecture's first download rule is: **do not relay large object downloads through your application server**.

If the backend fetches an object and copies every byte to the client:

- A server worker, goroutine, or event-loop task remains occupied during the transfer.
- Your application infrastructure handles bandwidth it does not need to handle.
- Cloud egress costs may increase.
- Downloads are often slower than delivery through a geographically distributed CDN.

Prefer direct object-storage delivery or a CDN in front of object storage.

---

9. Three download patterns

Pattern 1: public content, ideally through a CDN

For intentionally public assets (website images, CSS/JS bundles, public product images):

- A public bucket can work.
- A stronger common pattern is a private bucket with a CDN as the only reader.
- Set suitable cache-control headers.

- Use content-addressed keys (for example, a hash in the key) so a content change produces a new URL.

Benefits:

- High cache reuse.
- Fast delivery from edge locations near users.
- Long cache lifetimes are safer when object URLs change with content.

Pattern 2: pre-signed GET URLs

For private objects:

1. Client asks backend for access to a specific object.
2. Backend authenticates and authorizes the user.
3. Backend returns a short-lived pre-signed GET URL.
4. Browser downloads directly from object storage/CDN.

The lecture suggests a validity period such as 5–15 minutes, depending on product requirements.

The cache-key problem

A pre-signed URL contains a timestamp/signature. If every request produces a different URL, a CDN may treat each URL as a different cache key. The same underlying object then gets poor cache reuse.

Two approaches from the lecture:

- **Expiry bucketing:** sign URLs to expire at a shared boundary (for example, the top of the next hour), so users requesting the same object within that window can receive the same URL. This trades precise expiry for better cache reuse.
- **CDN-native signed URLs/cookies:** authorize at the CDN edge while letting the CDN manage its cache key and delivery behavior. Signed cookies are especially convenient for video sessions that request hundreds or thousands of segments.

Ensure that the CDN's cache-key and authorization configuration is designed deliberately; do not accidentally cache private content across users.

Pattern 3: CDN edge authorization

The CDN can validate signed URLs or cookies at edge locations. This moves authorization closer to the user and can preserve efficient caching.

For segmented video, one signed cookie can authorize a session to fetch many segment objects without the backend signing every segment URL individually.

---

10. HTTP range requests

A range request asks for only a byte interval of an object, rather than the entire object.

Example request:

```
GET /large-file
Range: bytes=1000000-3000000
```

A successful partial response uses:

- **206 Partial Content**
- `Content-Range` to identify the returned byte interval and total object size

If the requested range cannot be satisfied, the server can return:

- **416 Range Not Satisfiable**

Range requests are part of HTTP semantics and are supported by many object-storage services.

10.1 What range requests enable

Video seeking

- A player maps a desired time position to an approximate byte offset using the media index/container metadata.
- It requests bytes near that offset.
- It need not download all preceding bytes merely to jump forward.

Resumable downloads

- If a download stops at byte 400 million, the client can request the remaining range from that offset.
- It avoids retransmitting the bytes already received.

Parallel downloads

- A download manager can open multiple connections.
- Each connection requests a different byte range.
- The client combines the returned ranges into the final file.

Partial inspection

- A client can inspect a small header range (for example, the first few bytes) without downloading the whole object. The lecture demonstrates identifying a PNG header from a tiny range.

---

11. Range-based downloads versus segmented streaming

Range requests are powerful, but they still operate on one representation/version of an object. Modern video platforms often need multiple quality levels.

11.1 HLS and DASH workflow

The lecture describes a common segmented-streaming pipeline:

1. Start with an original uploaded video.
2. Transcode it into multiple quality/bitrate renditions.
3. Split each rendition into short segments.
4. Generate a manifest describing available renditions and segment locations.
5. Store the segments and manifest as objects.
6. The player fetches the manifest, then requests segments as playback proceeds.

11.2 Adaptive bitrate (ABR)

The player measures download performance while fetching segments:

- If the connection is fast enough, it can request a higher-quality segment.
- If the connection slows down, it can request a lower-quality segment.
- This helps playback continue while adapting visual quality to network conditions.

HLS and DASH are delivery formats/protocol ecosystems that support segmented streaming and adaptive bitrate behavior.

11.3 What object storage does

From the storage perspective:

- Each media segment is an object.
- The manifest is another object.
- A CDN can cache and deliver both.

The difficult engineering work is often the media-processing pipeline:

- Triggering jobs after upload
- Transcoding
- Segment generation
- Manifest generation
- Queue/worker orchestration
- Failure handling and retries

If video is a small feature of a product, a hosted video service may be more economical than building the whole pipeline. If video is the core product, owning the pipeline may make sense.

---

12. Object-storage costs

The lecture groups costs into three major categories.

Cost dimension	What it measures	Architectural implication
Storage	Amount of data stored over time (often GB-month)	Clean up abandoned data; choose retention and lifecycle policies
Operations	Number/type of API requests	Avoid unnecessary requests; account for write/read price differences
Egress	Data transferred out of the provider	Use CDN caching and compare provider egress pricing

The lecture notes that writes/mutations can be more expensive than reads in some pricing models, and highlights Cloudflare R2 as an example of a provider known for an egress-free pricing model. Pricing changes, so always check the provider's current pricing page before making a cost decision.

Cost-aware design choices

- Use lifecycle rules for incomplete multipart uploads.
- Avoid routing large downloads through application servers.
- Use CDN caching for repeatedly accessed objects.
- Choose cache keys and signed access mechanisms carefully.
- Avoid unnecessary bucket listing and redundant metadata calls.
- Evaluate storage, request, and egress costs together rather than optimizing only one.

---

13. End-to-end implementation checklist

Multipart upload

- ■ Validate user authorization, file size, content type, and allowed object destination.
- ■ Create multipart upload server-side and persist a pending database record.
- ■ Calculate part size with the 10,000-part cap in mind.
- ■ Use a bounded concurrency limit (start around 4–6 and measure).
- ■ Retain every successful part number and ETag.
- ■ Retry failed parts independently.

- Generate pre-signed part URLs in batches for very large uploads.
- Expose ETag through bucket CORS configuration.
- Complete the upload server-side.
- Verify the finalized object with HEAD and trusted checksum/metadata.
- Mark the database record completed only after verification.
- Abort cancelled uploads and configure an incomplete-upload lifecycle rule.

Download and delivery

- Do not proxy large file bytes through the application backend.
- Use public CDN delivery only for intentionally public content.
- Authorize private object access before issuing pre-signed GET access.
- Use CDN-native signed URLs/cookies where edge authorization is appropriate.
- Design CDN cache keys so signed access does not destroy cache reuse.
- Support range requests where clients need seeking or resumable downloads.
- Use HLS/DASH-style segmented delivery when adaptive video quality is required.
- Monitor storage, request, and egress costs separately.

---

14. Interview questions and answers

Q1. Why is a single PUT unsuitable for very large uploads?

A. It has a provider-enforced size ceiling, is all-or-nothing on interruption, can underutilize network throughput, and offers poor pause/resume and retry behavior.

Q2. What are the three main multipart-upload operations?

A. Create the multipart upload, upload each part, and complete the multipart upload.

Q3. What is an upload ID?

A. It identifies the in-progress multipart upload and its staged parts before the final object is committed.

Q4. Can multipart parts arrive out of order?

A. Yes. Each part request is independent; the final completion request supplies the ordered part-number/ETag list.

Q5. Why must the client retain ETags?

A. The completion request needs the ETag associated with each successfully uploaded part.

Q6. Why should part size be dynamic?

A. There is a maximum part count. Tiny fixed parts can exceed the 10,000-part limit for large files.

Q7. Why not use extremely large parts?

A. A failed part must be retransmitted in full, making retries expensive.

Q8. Can a multipart ETag be compared directly with the MD5 of the entire file?

A. Not generally. Multipart ETags have different semantics; use explicit provider checksums or a separately computed whole-file hash.

Q9. What happens to parts from an abandoned multipart upload?

A. They can remain stored and billable even though the final object is not visible in normal object listings. Configure an abort-incomplete-multipart lifecycle rule.

Q10. Why upload directly from the browser to object storage?

A. It avoids consuming backend workers and backend bandwidth for large byte transfers.

Q11. Why is ETag exposure in CORS important?

A. Browser JavaScript needs to read the ETag response header to submit the part list during completion.

Q12. How does multipart resume work?

A. List the already uploaded parts for the upload ID, then issue fresh URLs and upload only missing parts.

Q13. Why can pre-signed GET URLs reduce CDN cache hits?

A. Their changing signatures/timestamps can create different cache keys for the same object.

Q14. What are two ways to address the signed-URL cache issue?

A. Bucket expiries into shared time windows or use CDN-native signed URLs/cookies with deliberate cache-key behavior.

Q15. What does HTTP 206 mean?

A. Partial Content: the server is returning the requested byte range.

Q16. What does HTTP 416 mean?

A. Range Not Satisfiable: the requested range cannot be served.

Q17. How do range requests support resumable downloads?

A. The client requests bytes starting from the last successfully received offset.

Q18. What is adaptive bitrate streaming?

A. The player selects among encoded quality levels based on measured download conditions, usually segment by segment.

Q19. What is the difference between range-based delivery and HLS/DASH?

A. Range requests fetch byte intervals of a representation; HLS/DASH deliver separately encoded, segmented representations and manifests for adaptive playback.

Q20. What are the three major object-storage cost categories?

A. Stored data, API operations, and egress bandwidth.

---

15. Quick revision sheet

- Single PUT: size limit + all-or-nothing + throughput + UX problems.
- Multipart: create upload → upload parts → complete upload.
- Upload ID: staging identifier; final object appears after completion.
- Parts: independent, parallel, out of order, individually retryable.
- ETags: retain for completion; multipart ETag is not a reliable whole-file MD5.
- Part cap: 10,000; calculate part size dynamically.
- Retry granularity: one part; giant parts make retries expensive.
- Abandoned uploads: incomplete parts can still cost money; lifecycle abort rule (lecture default: 7 days).
- Browser architecture: backend authorizes and initiates; browser uploads directly to bucket; backend completes and verifies.
- Concurrency: start around 4–6 simultaneous part uploads; measure and tune.
- Browser memory: roughly part size × concurrent in-flight parts, plus implementation overhead.
- CORS: expose ETag so browser JavaScript can read it.
- Resume: list parts, reissue URLs for missing parts only.
- Downloads: do not proxy large files through backend.
- Public assets: private bucket + CDN + content-addressed keys is a strong pattern.
- Private assets: authorize, then use pre-signed GET or CDN signed access.
- Signed URL cache issue: varying signatures can fragment cache keys.
- Range: `206` + `Content-Range`; invalid range can yield `416`.
- Range enables seek, resume, and parallel downloads.
- HLS/DASH: transcode → segment → manifest → player adapts bitrate.
- Costs: storage + operations + egress.

---

16. Important distinctions to remember

Concept	Do not confuse it with
Upload ID	Final object key; the object is not committed yet
Part ETag	Guaranteed whole-file MD5
Multipart completion	Re-uploading all bytes through your backend
CORS	Authentication or authorization
Pre-signed URL	A permanent public URL
CDN cache key	The underlying object identity alone; query/signature handling matters
HTTP range request	HLS/DASH segmented adaptive streaming
Durability/storage	Free bandwidth or free API operations

Core mental model: *object storage should carry the file bytes directly. Your backend should control authorization, upload intent, metadata, completion, and application state—not act as a byte relay for every large transfer.*